LAB REPORT - 6

Problem 01: Aircraft INS Sensor Drift Prediction

> An Aircraft Inertial Navigation System (INS) relies on gyroscopes and accelerometers. Over time, these suffer from sensor drift error. We predict and compensate for gyroscope drift error (Y) by modeling it as a function of temperature (T), flight time (t), and vibration (V) using Multiple Linear Regression:
>
> $$\text{Drift} = a + b_1 T + b_2 t + b_3 V + \varepsilon$$
>
> The simultaneous influence of these variables is evaluated to find the specific impact of each condition on the sensor drift.

MATLAB Implementation Code:

```
% INS Sensor Drift Prediction (Multiple Linear Regression)
% Model Equation: Drift = a + b1(Temp) + b2(Time) + b3(Vibration) + error

clc; clear; close all;

% 1. Load Flight Test Empirical Data
temp_C    = [20, 25, 30, 35, 40, 45, 50, 55, 60, 65]';
time_hr   = [1, 5, 2, 8, 4, 10, 3, 7, 6, 9]';
vib_level = [2.1, 4.5, 3.2, 7.8, 5.6, 9.1, 3.9, 8.2, 6.5, 10.3]';
drift_deg = [0.8, 2.1, 1.5, 3.8, 2.7, 5.0, 2.0, 4.2, 3.4, 4.8]';

% 2. Visualize Individual Relationships
figure('Name', 'Drift Variable Analysis', 'Color', 'w');

subplot(1, 3, 1);
scatter(temp_C, drift_deg, 70, 'r', 'filled');
title('Drift vs Temperature'); xlabel('Temp (\circC)'); ylabel('Drift (\circ/hr)'); grid
on;

subplot(1, 3, 2);
scatter(time_hr, drift_deg, 70, 'g', 'filled');
title('Drift vs Flight Time'); xlabel('Time (hr)'); grid on;

subplot(1, 3, 3);
scatter(vib_level, drift_deg, 70, 'b', 'filled');
title('Drift vs Vibration'); xlabel('Vibration Level'); grid on;

% 3. Check Correlation Between Variables
fprintf('--- Variable Correlation Matrix ---\n');
corr_matrix = corrcoef([temp_C, time_hr, vib_level])

% 4. Build Multiple Regression Model
sensor_tbl = table(temp_C, time_hr, vib_level, drift_deg, ...
```

```
    'VariableNames', {'Temperature', 'FlightTime', 'Vibration', 'Drift'});
mdl = fitlm(sensor_tbl, 'Drift ~ Temperature + FlightTime + Vibration');

% 5. Extract and Display Coefficients
coeffs = mdl.Coefficients.Estimate;
fprintf('\n--- Final Regression Equation ---\n');
fprintf('Drift = %.4f + (%.4f * T) + (%.4f * t) + (%.4f * V)\n', ...
    coeffs(1), coeffs(2), coeffs(3), coeffs(4));

% 6. Model Evaluation Metrics
fprintf('\n--- Model Accuracy Metrics ---\n');
fprintf('R-Squared Value = %.4f\n', mdl.Rsquared.Ordinary);
fprintf('RMSE (Error)    = %.4f\n', sqrt(mean(mdl.Residuals.Raw.^2)));

% 7. Prediction for a New Flight Condition
new_test_data = table(55, 8, 9, 'VariableNames', {'Temperature', 'FlightTime',
'Vibration'});
predicted_drift = predict(mdl, new_test_data);
fprintf('\nPredicted Drift for [T=55, t=8, V=9] = %.4f deg/hr\n', predicted_drift);

% 8. Plot Actual vs Predicted Drift
figure('Name', 'Actual vs Predicted', 'Color', 'w');
plot(drift_deg, 'ko-', 'LineWidth', 1.5, 'MarkerSize', 6); hold on;
plot(predict(mdl, sensor_tbl), 'rx--', 'LineWidth', 1.5, 'MarkerSize', 6);
legend('Actual Empirical Data', 'Model Predictions', 'Location', 'best');
title('Multiple Linear Regression: Actual vs Predicted Drift');
xlabel('Data Sample Index'); ylabel('Drift Error (\circ/hour)');
grid on;
```

Simulation Output Plot:

Command Window

```
--- Variable Correlation Matrix ---

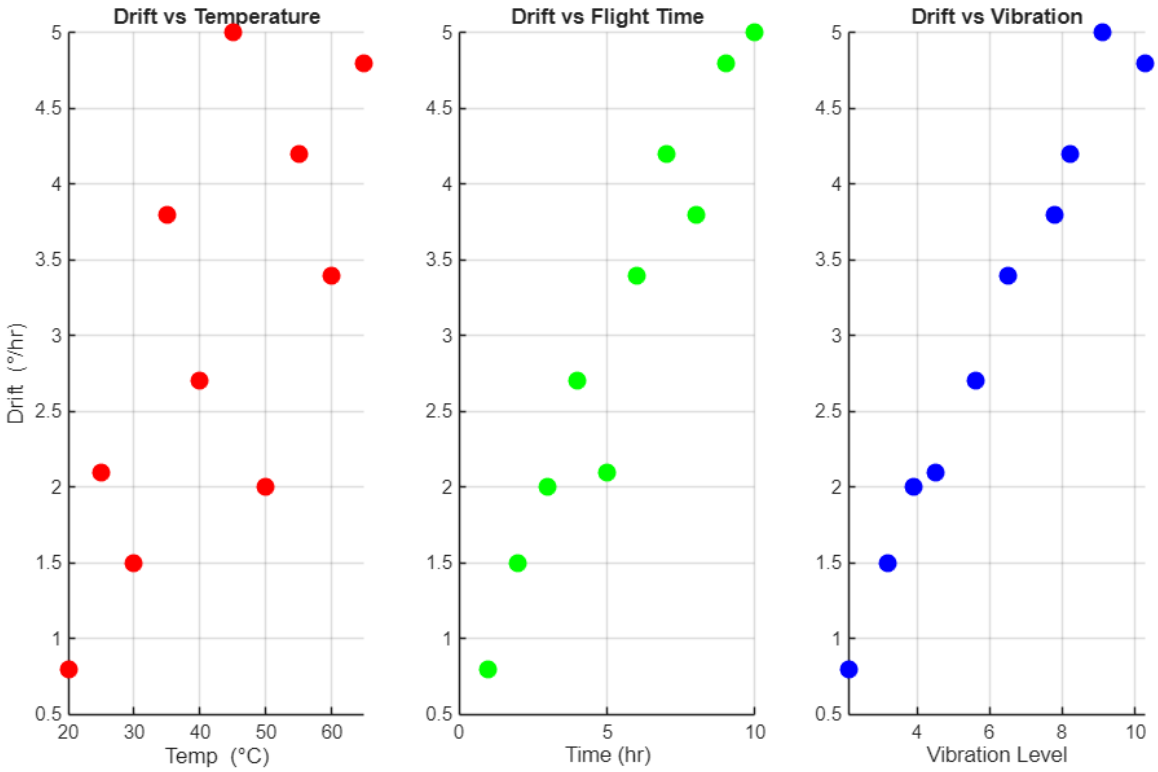
corr_matrix =

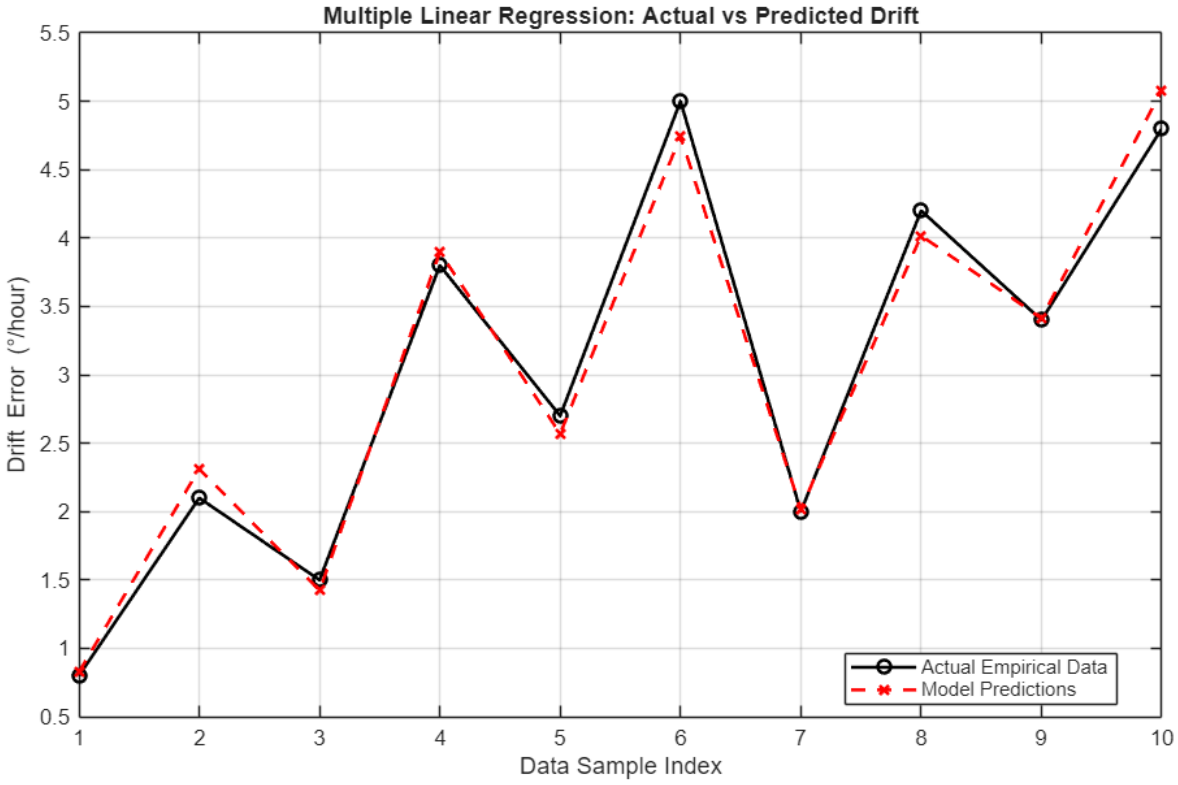
    1.0000    0.5758    0.7079
    0.5758    1.0000    0.9611
    0.7079    0.9611    1.0000


--- Final Regression Equation ---
Drift = -0.1457 + (0.0104 * T) + (0.1963 * t) + (0.2702 * V)

--- Model Accuracy Metrics ---
R-Squared Value = 0.9861
RMSE (Error)    = 0.1600

Predicted Drift for [T=55, t=8, V=9] = 4.4290 deg/hr
>>
```


Drift vs Temperature
Drift vs Flight Time
Drift vs Vibration
Drift (°/hr)
Temp (°C)
Time (hr)
Vibration Level


Multiple Linear Regression: Actual vs Predicted Drift
Drift Error (°/hour)
Data Sample Index
Actual Empirical Data
Model Predictions

Problem 02: Aircraft Radar Detection Range Prediction

> An aircraft's radar performance (received signal strength, S, in dBm) decreases as the target distance (D, in km) increases. Because the signal decays nonlinearly, a straight-line model is inaccurate.
>
> We apply a Nonlinear Regression Exponential Decay Model: $S(D) = \alpha \cdot e^{-\beta \cdot D}$
>
> Parameters α and β are optimized using nonlinear least squares algorithms (lsqcurvefit) to minimize the sum of squared differences between measured data and predicted values.

MATLAB Implementation Code:

```
% Radar Detection Range Prediction (Nonlinear Regression)
% Exponential Decay Model: S(D) = alpha * exp(-beta * D)

clc; clear; close all;

% 1. Experimental Radar Data
dist_km    = 10:10:100; % Target distance (km)
signal_dbm = [92, 78, 65, 54, 46, 38, 33, 28, 24, 20]; % Signal strength (dBm)

% 2. Initial Data Visualization
figure('Name', 'Radar Data', 'Color', 'w');
scatter(dist_km, signal_dbm, 80, 'k', 'filled'); grid on;
xlabel('Target Distance (km)'); ylabel('Received Signal Strength (dBm)');
title('Empirical Radar Signal Attenuation');

% 3. Model Definition & Nonlinear Fitting
% b(1) represents alpha, b(2) represents beta
decay_model = @(b, x) b(1) .* exp(-b(2) .* x);
initial_guess = [100, 0.05]; % Reasonable starting points

% Least squares curve fitting
options = optimset('Display', 'off');
params = lsqcurvefit(decay_model, initial_guess, dist_km, signal_dbm, [], [], options);

alpha_est = params(1);
beta_est  = params(2);

fprintf('--- Nonlinear Model Optimization ---\n');
fprintf('Alpha (\x03B1) = %.4f\n', alpha_est);
fprintf('Beta (\x03B2)  = %.6f\n', beta_est);

% 4. Evaluate Model Accuracy
pred_signal = decay_model(params, dist_km);
residuals = signal_dbm - pred_signal;
```

```
rmse = sqrt(mean(residuals.^2));

% Calculate R-squared for nonlinear fit
ss_tot = sum((signal_dbm - mean(signal_dbm)).^2);
ss_res = sum(residuals.^2);
r_squared = 1 - (ss_res / ss_tot);

fprintf('\n--- Performance Metrics ---\n');
fprintf('RMSE      = %.4f dBm\n', rmse);
fprintf('R-Squared = %.4f\n', r_squared);
fprintf('\nFinal Equation: S(D) = %.4f * exp(-%.6f * D)\n', alpha_est, beta_est);

% 5. Plot Fitted Decay Curve vs Actual Data
d_smooth = linspace(min(dist_km), max(dist_km), 200);
s_smooth = decay_model(params, d_smooth);

figure('Name', 'Fitted Model', 'Color', 'w');
scatter(dist_km, signal_dbm, 80, 'b', 'filled'); hold on;
plot(d_smooth, s_smooth, 'r-', 'LineWidth', 2); grid on;
xlabel('Target Distance D (km)'); ylabel('Signal Strength S (dBm)');
title('Nonlinear Regression: Radar Detection Range');
legend('Measured Data', 'Fitted Exponential Curve');

% 6. Model Extrapolation (Prediction)
test_dist = 120; % Predicting beyond our measured data
pred_val = decay_model(params, test_dist);
fprintf('\n--- Extrapolation ---\n');
fprintf('Predicted signal at %.1f km = %.4f dBm\n', test_dist, pred_val);
```

Simulation Output Plot:

Command Window

```
--- Nonlinear Model Optimization ---
Alpha (α) = 109.0926
Beta (β)  = 0.017178

--- Performance Metrics ---
RMSE      = 0.5520 dBm
R-Squared = 0.9994

Final Equation: S(D) = 109.0926 * exp(-0.017178 * D)

--- Extrapolation ---
Predicted signal at 120.0 km = 13.8850 dBm
>>
```

